Presentable Debugging with LLVManim

Benjamin Houle
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Benjamin_Houle@student.uml.edu

Ebenezer Oloyede
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Ebenezer_Oloyede@student.uml.edu

Lily Grippo
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Lily_Grippo@student.uml.edu

Mohammad Ahmed
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
ahmedzubair16dec@gmail.com

***Abstract*—LLVManim is a code visualization tool that generates aesthetically pleasing animations for information control flow and memory allocations. Utilizing the well-supported Manim library alongside LLVM's built-in static analysis tools and wide language support, LLVManim is able to quickly create informative, professionally styled animations for any code that compiles to LLVM IR. The primary objective of the project is to create an alternative to the unappealing "programmer art" and visual clutter that many standard tools suffer from, instead focusing on an output more suitable for presentations or educational purposes.**

CCS Concepts: **• Human-centered computing → Visualization → Visualization systems and tools → Visualization toolkits; • Software and its engineering → Software notations and tools → Compilers; • Applied computing → Education → Interactive learning environments**

***Index Terms*—program visualization, code animation, LLVM, static analysis, control flow graphs, memory visualization, educational tools, Manim, compiler infrastructure, presentation tools**

I. Introduction

Despite static analysis and data visualization being among the oldest concepts in computer science, it seems that disappointingly few graphic designers have been significantly interested in the former. While plenty of tools exist for creating control flow graphs or visualizing the stack, the majority of these favor a "function over form" paradigm that focuses on creating light-weight readable output, assuming the user is more concerned with the information than its presentation. This, however, is not always the case: presenting in any context, such as at a conference or school, generally favors quickly digestible and visually appealing content opposed to static, large blocks of text. In these cases, the presenter is currently forced to choose between using the unappealing generated visualization and the slog of creating something presentable for themselves. LLVManim provides a solution to this problem by utilizing the Manim library to create visually impressive animations from LLVM IR and runtime traces, allowing users to easily generate professional-quality visualizations without needing to invest significant time in manual design.

II. Existing Tools

A. PythonTutor

PythonTutor [1] is a widely used educational tool for visualizing program execution, originally developed to help novices build accurate mental models of how code runs. Its core contribution is a step-by-step execution trace that exposes stack frames, heap objects, and control flow in a concrete, visual manner. As described by its creator, Philip J. Guo, PythonTutor aims to "help people overcome a fundamental barrier to learning programming: understanding what happens as the computer runs each line of code" [2].

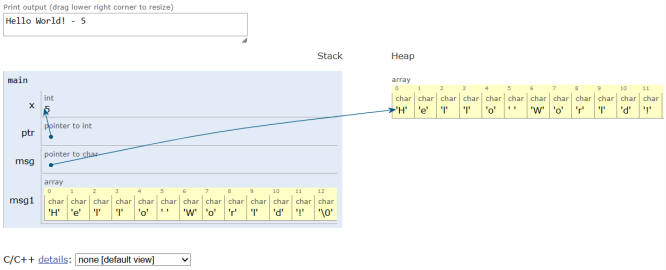

Fig. 1. Example PythonTutor.com output.

In fact, PythonTutor has been shown to be effective in improving students' understanding of programming concepts; Guo's further studies demonstrated significant learning gains when PythonTutor was used as a supplement to traditional instruction [3]. This shows the educational value of connecting code execution to visual representations, as well as the benefits of an accessible, interactive tool.

However, the visualizations produced by PythonTutor are relatively basic and not suitable for presentation purposes. PythonTutor's primary focus is on clarity and educational value rather than aesthetics, resulting in output that may be difficult to read or visually unappealing when projected to an audience. This, along with the fact that the tool is web-

based and not designed for offline use, makes it less than ideal for use in settings such as conference presentations where visual impact is important. PythonTutor is additionally restricted by its limited customization options, minimal library of visualization methods, and support for only C, C++, Java, Javascript, and Python.

B. LLVM Analysis and Transform Passes

LLVM has emerged as the de-facto standard compiler infrastructure for modern software development, powering everything from the development environment for Apple operating systems to cutting-edge systems research efforts. Its adoption by the compiler community can be attributed to several key strengths–a modular architecture that separates language frontends from optimization backends, an extensive suite of analysis and transformation passes that expose deep insights into program behavior, and robust support for over 30 programming languages, including those supported by PythonTutor. LLVM even provides users with an API to define their own language that compiles to its IR [4].

At the core of LLVM's value proposition for code analysis is its Intermediate Representation (IR), a low-level and platform-independent format that captures program semantics in a standardized way. This IR serves as the foundation for powerful built-in analysis passes that can generate control flow graphs, dominator trees, data flow analyses, and detailed memory access patterns. For compiler developers and researchers, these capabilities are invaluable—LLVM effectively solves the hard problem of extracting structured information from code, allowing them to focus on higher-level tasks like optimization or verification [5].

However, LLVM's visualization capabilities tell a different story. The standard approach for visualizing analysis results relies on generating DOT[1] files through passes like `-dot-cfg` or `-view-cfg`, which are then rendered using Graphviz [6]. While functionally correct, the resulting diagrams embody a strictly utilitarian aesthetic: monochrome nodes and edges, densely packed labels filled with LLVM IR instructions, and layouts optimized for completeness rather than clarity. This "function over form" philosophy serves well enough as a developer tool for debugging optimizer passes, but it fails catastrophically as a solution for communication beyond that narrow domain. A lecturer explaining recursion to undergraduates, a tech lead presenting architectural decisions to stakeholders, or a researcher delivering a conference talk all face the same problem: LLVM-generated visualizations are too dense, too technical, and frankly, too ugly to serve their pedagogical or presentational needs.

The barrier extends beyond mere aesthetics. Interpreting these diagrams requires familiarity with LLVM IR syntax—understanding what "`%4 = load i32, i32* %3`" means demands compiler knowledge that most audiences lack. The visualizations are fundamentally static images with

[1]Abstract grammar for defining Graphviz nodes, edges, graphs, subgraphs, and clusters.

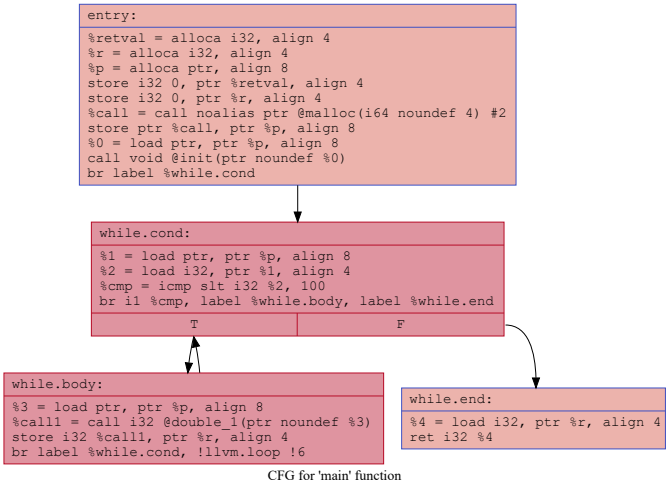

Fig. 2. Example Control-Flow Graph (CFG) using LLVM `-dot-cfg` pass. While technically accurate, the visualization prioritizes information density over visual clarity.

no mechanism for progressive disclosure or animation, forcing presenters to mentally simulate execution flow while pointing at a frozen graph. Moreover, customization options are severely limited; changing colors, adjusting layouts, or highlighting specific execution paths requires manual post-processing in external tools, completely defeating the purpose of automated generation.

In essence, LLVM provides extraordinarily powerful analysis capabilities but pairs them with visualization tools that assume their primary consumer is someone who already understands what they're looking at. This creates a frustrating gap: we can automatically extract rich semantic information about program behavior, but we cannot automatically present that information in a way that serves educational or presentational contexts. LLVManim aims to bridge this gap by maintaining LLVM's analytical rigor while replacing its austere output with animations designed for human communication rather than machine precision.

III. Manim

The Manim library [7] was originally developed by Grant Sanderson [8] for his popular educational YouTube channel "3Blue1Brown" to create animations of mathematical and computer science concepts. [9] His approachable animations have helped him become one of the most well-known online educators in these fields and have helped countless others understand complex topics more easily. Given this history, Manim represents an excellent candidate for creating high-quality educational presentations. Unfortunately, many potential users lack either the ability or the willingness to invest the substantial time required to create custom animations with this Python library.

While tools like PythonTutor and LLVM successfully address the technical challenge of extracting and displaying program information, they fall short in making that information

genuinely engaging. This is where Manim distinguishes itself, not as a code analysis tool, but as a demonstration of what becomes possible when visualization prioritizes both clarity and aesthetic appeal.

Since its inception in 2015, the 3Blue1Brown channel has gained more than 6 million subscribers and hundreds of millions of views. Individual videos on topics such as linear algebra, calculus, and even neural networks regularly reach audiences of millions. Sanderson's approach, using smooth, precisely choreographed animations to build intuition about abstract concepts, has earned him widespread recognition in the educational community. The key to all of these animations is Manim, an animation engine written in Python and maintained by Sanderson himself.

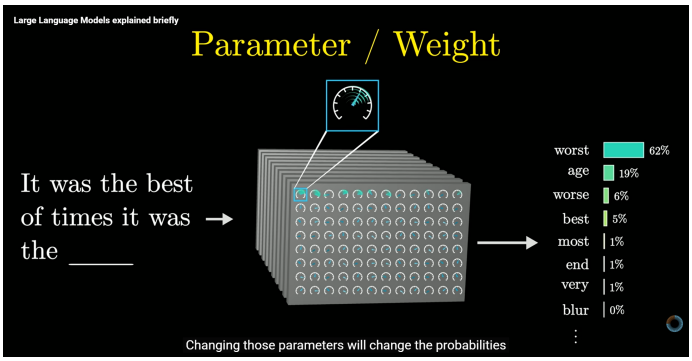

Fig. 3. An example of a Manim animation from the 3Blue1Brown YouTube video: Large Language Models explained briefly [10].

Manim's strength lies in its ability to programmatically generate animations with mathematical precision. Instead of tiresome keyframe animation like other software, Manim allows users to write Python code to determine how different mathematical objects should look, change, and interact with each other. Manim can render beautiful 3D scenes, render LaTeX equations, and output crisp 60 fps animations for any purpose, from YouTube tutorials to academic conferences. What makes Manim unique for education is its capacity to make complicated ideas, like matrix transformations, function composition, or even program flow, tangible and understandable to the viewer by systematically unfolding them.

Yet, this power comes with a barrier to entry. It is not trivial for a professor who wants to make a presentation for a single lecture, or a researcher who wants to create visualizations for a conference presentation, to learn and use Manim effectively. They may have a clear idea of what they want to show, perhaps how a recursive function builds up its stack, but writing Manim code for it requires a distinct skill set. LLVManim addresses this challenge by automatically translating program semantics into visual representations, thereby eliminating the barrier of manual coding between LLVM's analytical power and Manim's visual sophistication.

IV. LLVMANIM

By providing a solution to quickly and easily create data-based animations, LLVManim aims to bridge the gap between these two concepts:

1) Methods exist to quickly generate code visualizations, but they are unappealing for presentation purposes.
2) The Manim library can make visually impressive animations, but using it is time consuming.

Educators and tech leads alike can rejoice in spicing up their PowerPoints without needing to compromise between style and accessibility.

A. Project Goals

The primary goal of LLVManim is to create a lightweight pipeline for converting LLVM IR and runtime traces into concise, animated explanations of program behavior. As a proof-of-concept, the initial version of LLVManim will focus on visualizing fundamentals, such as control flow and memory allocations.

Proof-of-Concept Targets:

- **User Interface:** Develop a simple command-line interface that allows users to specify input files, select visualization options, and generate animations with minimal configuration.
- **LLVM IR Parsing:** Extract control flow and memory access information from LLVM IR, utilizing debug metadata when available.
- **Manim Animation Generation:** Translate LLVM IR and trace information into Manim scenes that visually represent control flow, stack frames, heap allocations, and data flow.
- **IR Instruction Coverage:** Support for the most common LLVM IR instructions, such as `load`, `store`, `call`, and basic arithmetic operations. This will allow LLVManim to visualize a wide range of simple programs.
- **Basic Animation Customization:** Allow users to customize the appearance of animations (e.g. color schemes, animation speed, etc.) to suit different presentation contexts.

Major features (required to realize the intended product):

- **Advanced Animation Customization:** Allow users to customize more aspects of the animations, such as layout styles, node shapes, or animation effects.
- **Basic Interactive Features:** Implement basic interactivity in the generated animations, such as pausing, stepping through execution, or highlighting specific paths.
- **Trace Event Mapping:** Correctly map $\geq 95\%$ of runtime events (loads, stores, branches, calls, returns) to IR locations when debug metadata is present.

Stretch features (nice-to-have if time permits):

- **GUI:** Create a graphical user interface for users who prefer not to use the command line.
- **Advanced Interactive Features:** Implement advanced interactivity in the generated animations, such as allowing users to click on nodes to see underlying IR code or variable values.

Deliverables:

- A command-line tool that accepts source files or LLVM IR as input and produces animations as output.
- A library of reusable Manim scene components for visualizing control flow graphs and memory states.
- Sample visualizations demonstrating LLVManim on standard programs.
- Documentation covering installation, usage, and supported IR constructs.

B. Implementation Plan

We approach the implementation of LLVManim as a 3-layer architecture: **Ingestion**, **Transformation**, and **Presentation**. The Ingestion layer collects LLVM IR and metadata. The transformation layer maps the collected IR constructs and trace events to an intermediate scene graph. The presentation layer renders the scene graph with Manim animations and exposes a small API for interactive playback.

1) Presentation Layer: The Presentation layer renders the scene graph using Manim and provides a simple interface for the user to render & view animations.

- **User Interface:** Command-line interface with automatic compilation:

```
# Visualize C program
$ llvmanim demo.c --mode=cfg --output=demo.mp4
# Custom speed, colors, & output format
$ llvmanim demo.c --mode=memory --speed=0.75
    ↪ --palette=dark --output=demo.gif
# Interactive mode
$ llvmanim demo.c --mode=trace --interactive
```

Listing 1. Mock CLI commands

  The CLI is intended for both developers integrating LLVManim into existing workflows and educators preparing course material. When the tool encounters unsupported IR constructs, missing debug metadata, or malformed input, it will emit a descriptive error message and continue processing the remainder of the program where possible.
- **Customization API:** Provide configuration options using command line flags and YAML files:
  - `-speed <factor>`: Animation playback speed
  - `-palette <name>`: Color scheme (dark, light, colorblind)
  - `-mode <type>`: Visualization mode (cfg, memory, trace)
  - `-verbose`: Include IR instruction text
- **Manim Code Generation:** Translate scene graph into Python code defining a Manim `Scene` class. Visual elements become `Mobject` instances with animation commands.
- **Rendering Pipeline:** Invoke Manim to produce supported output formats (MP4, MOV, WEBM, & GIF) or enable interactive openGL preview with keyboard controls.

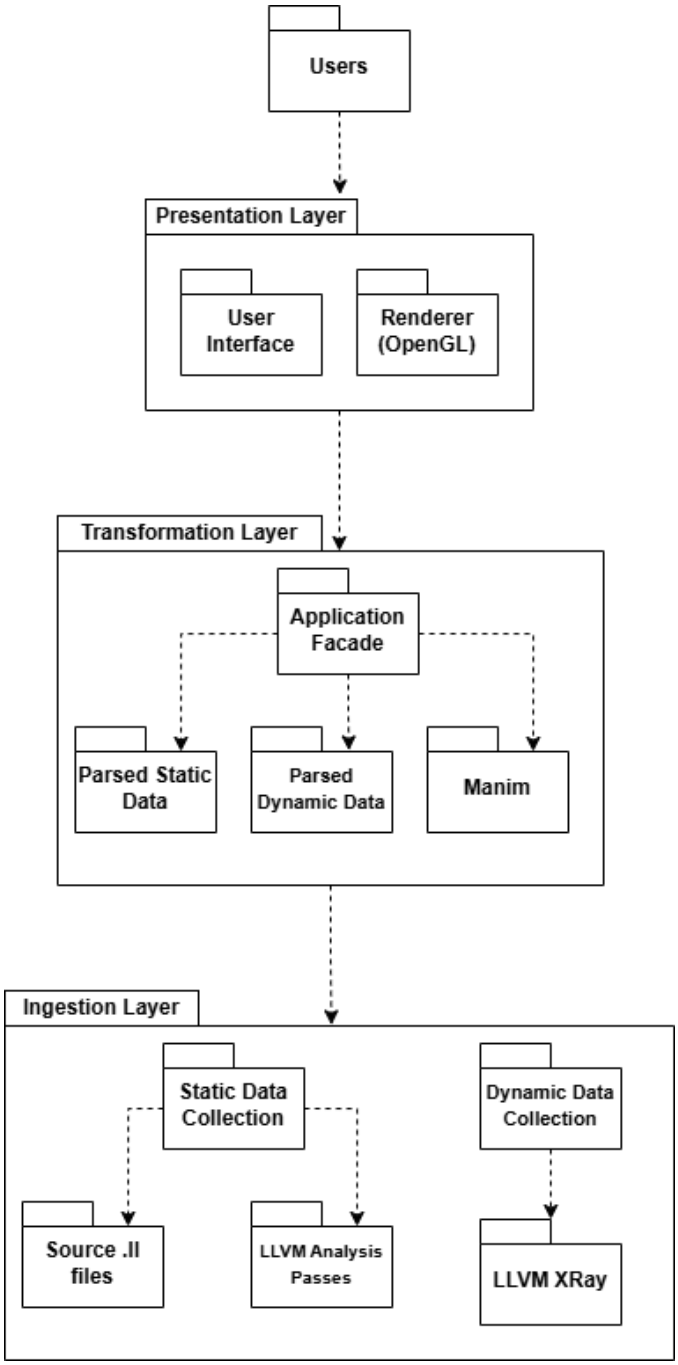

Fig. 4. Model diagram illustrating the three-layer architecture of LLVManim. The Ingestion layer collects LLVM IR and metadata, the Transformation layer maps it to an intermediate scene graph, and the Presentation layer renders it with Manim animations.

2) Transformation Layer: The Transformation layer maps LLVM constructs to an intermediate scene graph for Manim rendering.

- **Instruction-to-Animation Mapping:** Define animation primitives for each LLVM instruction:
  - `alloca`: Create stack frame with variable label
  - `load`/`store`: Animate arrow to/from memory location
  - `call`: Push new stack frame with fade-in
  - `ret`: Pop stack frame with fade-out
  - `br`: Highlight control flow edge with color change
- **Scene Graph Construction:** Create hierarchical representation where nodes correspond to visual elements (basic blocks, stack frames, memory cells) with properties for position, appearance, and timing.
- **Input:** Structured representation of program control flow and memory operations from the Ingestion layer.
- **Output (Application Facade):** Generated Manim instructions, sent to the Presentation layer for rendering.
- **(Optional) Timing Coordination:** Use parsed profiling data to calculate timing offsets for animation events (user-configurable).

3) Ingestion Layer: The Ingestion layer extracts program structure from LLVM IR. We use the `LLVMlite` [11] Python binding to parse .ll files generated by clang with the `-g` flag for debug metadata.

- **Static Data Collection:**
  - *Static Data Extraction:* Load LLVM modules and extract functions, basic blocks, instructions, and metadata using `LLVMlite`.
  - *Control Flow Extraction:* Use LLVM built-in analysis passes (e.g. `-opt`) to extract control-flow data from the IR.
- **Input:** Source files (automatically compiled to LLVM IR) or .ll files directly.
- **Output:** Structured representation of the program's control flow and memory operations for the Transformation layer to convert into a Manim scene.
- **(Optional) Dynamic Data Collection:** Extract instruction timing data from LLVM profiling tools, such as `LLVM-XRay` [12], to enable performance-based animation timing.

LLVManim will target a well-defined subset of LLVM IR, including standard control flow constructs (branches and loops), common memory operations (`alloca`, `load`, and `store`), and basic arithmetic and comparison instructions. Unsupported constructs like intrinsics, exception handling, or vector operations will be skipped with a warning message indicating which instructions were omitted, keeping the tool usable even on programs that use unsupported features.

V. Risk Evaluation

A. Technical Risks

- **LLVM IR Complexity:** LLVM IR supports a wide range of constructs, including vector operations, intrinsics, and optimization-introduced transformations. LLVManim targets a common subset of instructions for the initial implementation.
  - *Risk:* Unsupported constructs may cause parsing failures or incomplete visualizations.
  - *Mitigation:* Restrict the MVP to unoptimized `-O0` IR and provide clear diagnostic messages when encountering unsupported instructions.
- **Manim Performance:** Manim provides high-quality animations but can be computationally expensive, especially for scenes with many objects or transitions.
  - *Risk:* Long render times may slow development and limit iterative testing.
  - *Mitigation:* Use low-quality preview rendering during development and reserve high-quality output for final deliverables.
- **Cross-Platform Compatibility:** Achieving consistent LLVManim behavior across operating systems and development environments can be challenging.
  - *Risk:* Platform-specific issues may arise and cause inconsistent behavior or failures.
  - *Mitigation:* Use cross-platform libraries and conduct testing on multiple platforms to ensure compatibility.
- **Dependency Fragility:** LLVManim depends on Clang, LLVMlite, and Manim, each of which may introduce version incompatibilities or platform-specific issues.
  - *Risk:* Changes in LLVM or Manim APIs could break LLVManim's functionality.
  - *Mitigation:* Provide a reproducible environment via containerization or a standardized setup script.

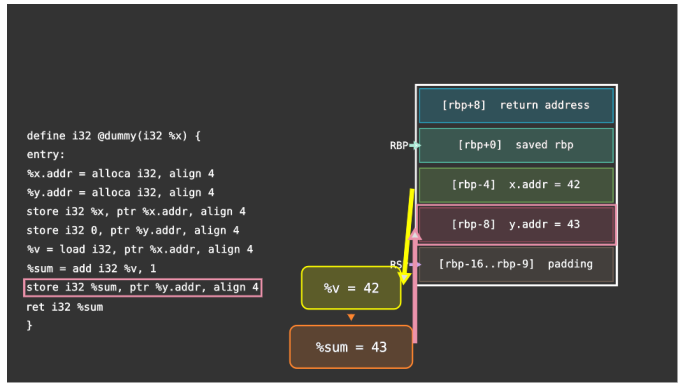

Fig. 5. Mock output of LLVManim showing a basic increment operation.

B. Architectural Risks

- **Ambitious Feature Set:** The proposal includes core, major, and stretch features. Several planned capabilities, including interactive stepping, advanced customization, and trace mapping above 95%, are substantial undertakings.
  - *Risk:* Delivering the complete feature set within the available project timeline may be difficult.

- *Mitigation:* Prioritize an MVP pipeline that covers IR ingestion, scene graph construction, and basic animation before implementing advanced capabilities.

- **User Expectations for Customization:** Educators and presenters often need fine-grained control over pacing, color schemes, and layout choices.
  - *Risk:* Limited customization may reduce adoption and may fail to meet expectations for presentation-quality output.
  - *Mitigation:* Provide a minimal but meaningful set of customization flags (e.g., palette, speed, verbosity) in the initial release.

Projected Timeline

Week	Task
W_1 $[2/08 - 2/14]$	• Project proposal draft complete. • Development environment setup.
W_2 $[2/15 - 2/21]$	• LLVM Parsing
W_3 $[2/22 - 2/28]$	• Project Architecture Revision • Stack Frame Animation
W_4 $[3/01 - 3/07]$	• Project Repository Submission • Mohammad/Ben: Parse LLVM IR with llvmlite; extract functions, basic blocks, instructions. • Lily/Ebenezer: Build clean internal program model (CFG + metadata).
W_5 $[3/08 - 3/14]$	• Ebenezer/Lily: Implement CLI with automatic clang compilation. • Mohammad/Ben: Convert CFG data into initial scene graph structure.
W_6 $[3/15 - 3/21]$	• Ben: Finalize scene graph design (nodes, edges, animation events). • Implement layout algorithm and timing coordination.
W_7 $[3/22 - 3/28]$	• Ben: Generate Manim code for CFG mode. • Lily: Deliver first end-to-end rendered animation.
W_8 $[3/29 - 4/04]$	• Ebenezer: Extend scene graph for stack and memory modeling. • Animate alloca, load, store, call, ret.
W_9 $[4/04 - 4/10]$	• Mohammad: Implement trace mode with instruction highlighting and speed control.
W_{10} $[4/11 - 4/17]$	• All: Add advanced customization (palette, layout, YAML) and pause/step interactivity. • ALL; integrate stretch feature (lightweight GUI or IR inspection), finalize docs and demos.

TABLE I
PROJECTED TIMELINE FOR LLVMANIM DEVELOPMENT.

Acknowledgment

We would like to thank Dr. James Daly for his guidance, feedback, and support throughout the development of this work.

References

[1] P. Guo, "Python Tutor: Visualize Python, Java, C, C++, and JavaScript code execution," https://pythontutor.com/, 2024, accessed: 2024-02-24.

[2] P. J. Guo, "Online python tutor: embeddable web-based program visualization for cs education," in *Proceeding of the 44th ACM Technical Symposium on Computer Science Education*, ser. SIGCSE '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 579–584. [Online]. Available: https://doi.org/10.1145/2445196.2445368

[3] ——, "Codeopticon: Real-time, one-to-many human tutoring for computer programming," in *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*, ser. UIST '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 599–608. [Online]. Available: https://doi.org/10.1145/2807442.2807469

[4] C. Lattner and V. Adve, "LLVM: A compilation strategy for lifelong program analysis and transformation," in *Proceedings of the 2nd IEEE Annual CGO and ACM SIGPLAN Annual CC Symposium*, ser. CGO-CC '04. Palo Alto, CA, USA: IEEE Computer Society, 2004, pp. 60–72.

[5] LLVM Project, *LLVM Language Reference Manual*, LLVM Project, 2026, accessed: Feb. 11, 2026. [Online]. Available: https://llvm.org/docs/LangRef.html

[6] J. Ellson, E. Gansner, E. Koutsofios, S. North, and G. Woodhull, "Graphviz - Open Graph Visualization Software," https://graphviz.org/, 2001, accessed: [Insert Date Here].

[7] G. Sanderson, "manim," 2015, accessed 2026-02-24. [Online]. Available: https://github.com/3b1b/manim

[8] ——, "3blue1brown," Website, 2026, accessed: Feb. 13, 2026. [Online]. Available: https://www.3blue1brown.com/

[9] ——, "3blue1brown," Youtube Channel, March 2015, accessed: Feb. 13, 2026. [Online]. Available: https://www.youtube.com/@3blue1brown

[10] 3Blue1Brown, "Large language models explained briefly," https://www.youtube.com/watch?v=LPZh9BOjkQs, 2023, accessed: 2026-02-24.

[11] Numba Development Team, "llvmlite: A lightweight llvm python binding for writing jit compilers," https://github.com/numba/llvmlite, 2024, accessed: 2026-02-25.

[12] LLVM Project, "Xray: Function call tracing and analysis," https://llvm.org/docs/XRay.html, 2024, accessed: 2026-02-25.

FEEDBACK

[1]— *"The figures are helpful for showing how the existing tools work, but for the proposal make sure you show how you want the visualized code to work. This could be through making a mockup or setting up input for Manim to generate something that looks like what you want. You should also think about the pipeline for what data you are planning on getting from LLVM, how you are going to get it, and how you will feed it into Manim."*

[2]— *"You have figures for some existing tools. Make sure you talk about them in the proposal and what their limitations are."*

---

[3]— *"This [PythonTutor] figure is unreadable."*

[4]— *"[PythonTutor] Spelling."*

[5]— *"Which languages [are supported by PythonTutor]?"*

[6]— *"You should have a citation for where to access the [PythonTutor] tool itself and not just the paper describing its efficacy."*

[7]— *"The text in the [control-flow graph] figure is too small."*

[8]— *"[Graphviz] Linky please."*

[9]— *"Linky to Manim."*

[10]— *"citation needed [for 3Blue1Brown's winning a Streamy award]"*

[11]— *"The order of the [model] diagram is in the opposite order that they are presented in [implementation plan] text."*

[12]— *"The items in the timeline are incredibly brief and need more detail or support."*

---

[13]— *"The figures are quite shrunk down especially figure 1 and figure 2 making it extremely difficult to read the content within it, if those were made larger it would greatly improve the readability."*

[14]— *"The abstract could briefly mention the intended users explicitly, such as educators, students, or software engineers. Additionally, the abstract could briefly summarize how LLVManim achieves this goal rather than only stating the objective."*

The intended users of LLVManim is a broad spectrum, including educators, students, and software engineers. We believe the abstract already addresses the context in which LLVManim is useful, and to specify the users that exist within those contexts would be redundant.

[15]— *"The introduction could more explicitly define the scope of the LLVManim system itself earlier in the section. Currently, LLVManim is only briefly mentioned at the end. Introducing LLVManim sooner would improve clarity and help frame the remainder of the document."*

[16]— *"[The PythonTutor] section could be slightly condensed. Some explanations regarding educational benefits are repeated, and the limitations relevant specifically to LLVManim's goals like their lack of customization, presentation-quality output, etc., should be emphasized more clearly. Additionally, explicitly comparing PythonTutor directly to LLVManim in one or two sentences would make the relevance stronger."*

We believe the current level of detail in the PythonTutor section is appropriate for providing necessary context and motivation for LLVManim. We may provide a more explicit comparison to LLVManim in the future, but we feel the current description sufficiently highlights the limitations of PythonTutor that LLVManim aims to address.

[17]— *"[The LLVM Analysis and Transform Passes] section could be improved by separating the strengths and weaknesses into clearly defined subsections or paragraphs. Currently, strengths and limitations are interwoven, which makes it slightly harder to skim. Additionally, some technical explanations like the IR syntax examples could be shortened slightly, as the focus of the proposal is visualization rather than compiler internals."*

We believe the current structure of the LLVM Analysis and Transform Passes section effectively conveys the necessary information without overcomplicating the overall document structure. It follows the same format as the other sections describing existing tools, and we feel it is consistent with the overall style of the proposal.

[18]— *"[The Manim] section could more explicitly explain how Manim integrates into the LLVManim system pipeline. Additionally, some historical background about 3Blue1Brown could be slightly condensed to maintain focus on the project itself."*

We reduced the historical background about 3Blue1Brown. However, these sections are focused on providing context for the tools we are using, and we feel that the current level of detail is appropriate.

[19]— *"Some proof-of-concept metrics (such as $\geq 95\%$ trace mapping and $\geq 80\%$ usability success rate) could briefly explain how these metrics will be measured or evaluated. This would strengthen the feasibility and evaluation clarity. Additionally, the distinction between core features and major features could be clarified slightly, as both appear required for the full system."*

[20]— *"The architecture diagram should be explicitly referenced and described more clearly in the text. Explaining what each arrow and*

component represents would improve clarity. Additionally, some implementation details (such as exact timing offsets and layout algorithms) may be overly detailed for a proposal and could be slightly condensed."

[21]— *"The output format could be explained more clearly, particularly how it connects to the Transformation layer."*

[22]— *"The scene graph structure could benefit from a brief visual example or mock representation to clarify how data flows through the system. Additionally, the modified Sugiyama layout algorithm is mentioned but not briefly explained, which may confuse readers unfamiliar with graph layout techniques."*

[23]— *"The relationship between scene graph generation and Manim output could be explained more clearly in terms of data flow. Additionally, it could briefly explain expected output formats and user workflows."*

[24]— *"[The User Interface] section could briefly explain error handling and limitations more clearly, such as unsupported language features or missing debug metadata. Additionally, this section could clarify whether the CLI is intended for developers, educators, or both."*

[25]— *"Testing and validation phases could be more clearly defined, particularly usability testing and feature validation. Additionally, stretch feature implementation is only briefly mentioned and could be explicitly identified in the timeline."*

We have included testing and validation as part of the overall implementation plan, but we have not explicitly defined separate phases for these activities. We will be continuously testing and validating our implementation throughout the development process, and we will prioritize core features before stretch features to ensure we have a MVP by the end of the semester.

[26]— *"The feedback section [...] could be improved by explicitly explaining how each feedback item will be addressed moving forward rather than only stating that it remains unaddressed. Additionally, creating mockups or example animation output would significantly strengthen the proposal and demonstrate feasibility."*

We have addressed feedback in the manner prescribed by the project guidelines, which is to either address it or explain why it remains unaddressed.

[27]— *"Ensure consistent formatting across all references and verify citation formatting matches IEEE or required course format exactly."*

We have reviewed our references and believe they are formatted correctly according to the IEEE style. If there are any specific formatting issues, we would appreciate further clarification so we can address them appropriately.

---

[28]— *"It's unnecessary to explain how Julia works. It distracts from the main focus of the paper."*

We do not explain how Julia works in the proposal. We have reworded the relevant section for clarity.

[29]— *"You don't have to include all these accomplishments. We already get the idea that [3Blue1Brown] is popular and well respected from earlier."*

[30]— *"Including pictures of Manim diagrams here would illustrate your point better."*

[31]— *"Your goals seem overly ambitious for just a one semester student project."*

We acknowledge that our goals are ambitious, but we believe they are achievable with careful planning and prioritization. We also believe that our approach to the project is more relevant to the goals of the class than the final output, and we are committed to making significant progress towards our objectives regardless of the final scope of the project.

[32]— *"It's unrealistic to expect work to be done over spring break."*

We appreciate the concern, but our teammembers are aware of the timeline and have planned accordingly.

[33]— *"You should talk more about the potential risks. It's not really mentioned at all."*

[34]— *"The motivation section could be strengthened. While the proposal explains that existing tools are either too technical or not presentation-friendly, it does not fully explain why this problem is important at a broader level. It would help to more clearly identify who specifically benefits from this tool and why current alternatives are insufficient. The proposal currently reads as though the improvement is mostly aesthetic, so emphasizing the educational or practical impact would make the motivation more compelling."*

We believe the motivation section sufficiently explains the importance of the problem and the intended beneficiaries of LLVManim. We feel that the educational and practical impact of LLVManim is clear from the context provided, and, frankly, the improvement is primarily aesthetic.

[35]— *"[some objectives] could be more measurable. For example, the proposal mentions performance and trace mapping goals, but it would be helpful to specify what constitutes success more concretely. Adding clearer success criteria would make it easier to evaluate whether the project meets its goals."*

[36]— *"The [architecture] scope could be clarified further. It would be helpful to specify what*

subset of LLVM IR will be fully supported and how unsupported constructs will be handled. Addressing potential edge cases would make the approach feel more complete and realistic."

[37]— *"The proposal would benefit from a clearer description of final deliverables. While features are described, it is not entirely clear what concrete artifacts will be submitted at the end of the project. Explicitly listing tangible deliverables would strengthen this section."*

[38]— *"The timeline is present but somewhat broad in places. Some weekly goals are described generally, which makes it difficult to determine what measurable progress will be achieved each week. Making each milestone more concrete and demonstrable would better align with the rubric's expectations."*

[39]— *"The proposal mentions evaluation goals such as trace mapping percentages and usability testing, which is good. However, the evaluation methodology could be described in more detail. For example, specifying the number of participants in the pilot study, the types of tasks they will perform, and how results will be measured would make the evaluation plan stronger and more rigorous."*

[40]— *"A clearly labeled risks section would improve the proposal. While potential challenges are implied, explicitly identifying technical and timeline risks and describing mitigation strategies would demonstrate thoughtful project planning."*

---

[41]— *"I feel [the PythonTutor section] should include the supported languages of PythonTutor, since the usefulness of the tool depends heavily on what languages are supported. It is more useful if it supports languages that are commonly used for education vs if it supports languages that are more obscure."*

[42]— *"What are DOT files? does this mean unix dot files, or does this stand for something?"*

[43]— *"What does verbosity mean in this application and how is it different from animation speed. Does the creator of the animation include an audio track, or is it the speed that words appear? If the latter, how and why are words appearing?"*

Verbosity in this context refers to the amount of information displayed in the animation, such as whether variable values are shown or only control flow is visualized. It is different from animation speed, which controls how quickly the animation plays. This is a common feature in debugging tools, and we feel it is reasonable to expect readers to understand this distinction.

[44]— *"Go more in depth about what the study entails. Will you be comparing against other similar tools, or just compared to only reading the code? Will it be between subjects or within subjects? Are you only going to collect data about accuracy or will you also collect data about subjective user preference? Also this study does not seem like it connects well to the stated goal of this project. You are discussing making a tool that will help educational scenarios, but are only evaluating it for use in debugging. I would add a component focusing on user experience generating the images, or evaluating which ones help people to better understand the purpose of code. There are also no sample graphs or tables for the data that is to be collected."*

[45]— *"Why separate core features from MVP features when core features are defined as "required to have a usable product at all", which is essentially the MVP?"*

[46]— *"How are animations utilized. Is it moving arrows to demonstrate code flow? Will sections of the code be moved on screen only when relevant, and removed afterward. For someone unfamiliar with Manim this is unclear."*

A major portion of this project will be dedicated to exploring precisely this question. While we have some initial mockups and ideas for how the animations will work, these likely will not be finalized until towards the end of the project.

[47]— *"While I understand that this document is written with the assumption that the reader has a computer science bachelors degree, I think it would help to use the long form of this abbreviation at least once, as was done with LLVM IR."*

[48]— *"Why make your own "JSON-like" format instead of just using standard JSON. If it is for data streaming purposes, perhaps consider JSON Lines."*

[49]— *"No discussion of risks or costs."*

[50]— *"Timeline does not say when the experiment will be performed."*

[51]— *"Missing time spent on project."*

[52]— *"The lack of Manim examples or proposed examples for this project makes it difficult for someone unfamiliar with that tool to understand what this tool will produce."*

HOURS SPENT: ~40 Hours